Le développement logiciel à l'ère de l'IA

Cours 4 — D'un agent à plusieurs, et l'avenir du développement logiciel

Pierre Sutra

Télécom SudParis

version 1.0

Rappel : le cours 3

- un **agent** = un LLM + des outils + une boucle, dans un *harness*
- l'autonomie est un **cadran** : plan, laisse courte, longue itération, YOLO
- l'**oracle** définit « fini » : sans tests, laisse courte ; avec de bons tests, longue itération
- git est le filet de sécurité : une tâche, une branche ; relire le *diff*
- le contexte est fini, la compaction est avec perte ; `AGENTS.md` et les *skills* guident l'agent

D'un agent à plusieurs

De délégant à concepteur.

- les limites rencontrées au cours 3 :
  - le contexte se remplit, et la compaction perd des détails
  - un seul agent cumule les rôles : il écrit le code *et* les tests qui le jugent
  - toute la vérification repose sur des tests

D'un agent à plusieurs

De délégant à concepteur.

- les limites rencontrées au cours 3 :
  - le contexte se remplit, et la compaction perd des détails
  - un seul agent cumule les rôles : il écrit le code *et* les tests qui le jugent
  - toute la vérification repose sur des tests
- aujourd'hui, trois axes :
  1. **plus d'agents** : une équipe, que vous dirigez depuis les bords de la boucle
  2. **de meilleurs oracles** : la machine spécifie, un vérificateur prouve
  3. **votre place** : quelle plus-value, quand la machine écrit le code ?

1. Des équipes de développeurs aux équipes d'agents
2. Les systèmes multi-agents : coordonner, communiquer
3. Les méthodes formelles : vérifier ce qu'écrit la machine
4. Votre plus-value, quand la machine écrit le code
5. Synthèse

- une personne maîtrise tout le cycle de vie : la conception, le code (souvent en assembleur), la mémoire, les graphismes



Apple IIe (1983)
photo : Mystère Martin, CC BY-SA 3.0

Les années 1980 : l'artisan

1. Des équipes de développeurs aux équipes d'agents

- une personne maîtrise tout le cycle de vie : la conception, le code (souvent en assembleur), la mémoire, les graphismes
- par exemple, les jeux vidéo : J. Mechner (*Karateka*, *Prince of Persia*) et R. Garriott (*Ultima*) écrivent les leurs presque seuls



Apple IIe (1983)
photo : Mystère Martin, CC BY-SA 3.0

Les années 1980 : l'artisan

1. Des équipes de développeurs aux équipes d'agents

- une personne maîtrise tout le cycle de vie : la conception, le code (souvent en assembleur), la mémoire, les graphismes
- par exemple, les jeux vidéo : J. Mechner (*Karateka*, *Prince of Persia*) et R. Garriott (*Ultima*) écrivent les leurs presque seuls
- une machine isolée, des ressources rares, pas de réseau



Apple IIe (1983)
photo : Mystère Martin, CC BY-SA 3.0

Les années 1980 : l'artisan

1. Des équipes de développeurs aux équipes d'agents

- une personne maîtrise tout le cycle de vie : la conception, le code (souvent en assembleur), la mémoire, les graphismes
- par exemple, les jeux vidéo : J. Mechner (*Karateka*, *Prince of Persia*) et R. Garriott (*Ultima*) écrivent les leurs presque seuls
- une machine isolée, des ressources rares, pas de réseau
- la maîtrise de bout en bout était possible : **le système tenait dans une seule machine**

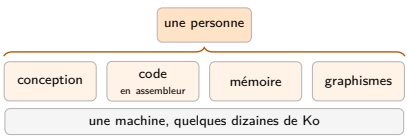


Apple IIe (1983)
photo : Mystère Martin, CC BY-SA 3.0

- une personne maîtrise tout le cycle de vie : la conception, le code (souvent en assembleur), la mémoire, les graphismes
- par exemple, les jeux vidéo : J. Mechner (*Karateka*, *Prince of Persia*) et R. Garriott (*Ultima*) écrivent les leurs presque seuls
- une machine isolée, des ressources rares, pas de réseau
- la maîtrise de bout en bout était possible : **le système tenait dans une seule machine**



Apple IIe (1983)
photo : Mystère Martin, CC BY-SA 3.0



Les années 1990-2000 : les silos

1. Des équipes de développeurs aux équipes d'agents

- le Web, les bases de données relationnelles, les architectures client-serveur puis *n*-tiers



serveurs Sun Enterprise 3000 et baie de stockage A5000
(fin des années 1990)
photo : Wolfgang Stief, CC0

Les années 1990-2000 : les silos

- le Web, les bases de données relationnelles, les architectures client-serveur puis *n*-tiers
- le code tourne sur un réseau : personne n'est expert à la fois de l'infrastructure, de la base et de l'interface



serveurs Sun Enterprise 3000 et baie de stockage A5000
(fin des années 1990)
photo : Wolfgang Stief, CC0

Les années 1990-2000 : les silos

- le Web, les bases de données relationnelles, les architectures client-serveur puis *n*-tiers
- le code tourne sur un réseau : personne n'est expert à la fois de l'infrastructure, de la base et de l'interface
- la réponse : des **silos**. Le développeur (Dev) écrit le code ; l'administrateur système (Ops) le déploie ; l'administrateur de la base (DBA) gère les données



serveurs Sun Enterprise 3000 et baie de stockage A5000
(fin des années 1990)
photo : Wolfgang Stief, CC0

Les années 1990-2000 : les silos

- le Web, les bases de données relationnelles, les architectures client-serveur puis *n*-tiers
- le code tourne sur un réseau : personne n'est expert à la fois de l'infrastructure, de la base et de l'interface
- la réponse : des **silos**. Le développeur (Dev) écrit le code ; l'administrateur système (Ops) le déploie ; l'administrateur de la base (DBA) gère les données
- entre les silos, des tickets : le code est « jeté par-dessus le mur »



serveurs Sun Enterprise 3000 et baie de stockage A5000
(fin des années 1990)
photo : Wolfgang Stief, CC0

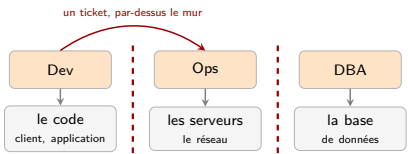
Les années 1990-2000 : les silos

1. Des équipes de développeurs aux équipes d'agents

- le Web, les bases de données relationnelles, les architectures client-serveur puis *n*-tiers
- le code tourne sur un réseau : personne n'est expert à la fois de l'infrastructure, de la base et de l'interface
- la réponse : des **silos**. Le développeur (Dev) écrit le code ; l'administrateur système (Ops) le déploie ; l'administrateur de la base (DBA) gère les données
- entre les silos, des tickets : le code est « jeté par-dessus le mur »



serveurs Sun Enterprise 3000 et baie de stockage A5000
(fin des années 1990)
photo : Wolfgang Stief, CC0



Les années 2010-2020 : l'hyperspécialisation

1. Des équipes de développeurs aux équipes d'agents

- le *cloud*, les microservices, l'intégration et le déploiement continus



un *datacenter* de *cloud*
photo : Infopro Digital, droits réservés

Les années 2010-2020 : l'hyperspécialisation

1. Des équipes de développeurs aux équipes d'agents

- le *cloud*, les microservices, l'intégration et le déploiement continus
- les rôles éclatent : *frontend*, *backend*, mobile, DevOps, SRE, données, QA, SecOps ; côté produit, *Product Owner*, UX, *Scrum Master*



un *datacenter* de *cloud*
photo : Infopro Digital, droits réservés

Les années 2010-2020 : l'hyperspécialisation

1. Des équipes de développeurs aux équipes d'agents

- le *cloud*, les microservices, l'intégration et le déploiement continus
- les rôles éclatent : *frontend*, *backend*, mobile, DevOps, SRE, données, QA, SecOps ; côté produit, *Product Owner*, UX, *Scrum Master*
- la loi de Conway † : un système copie la structure de communication de l'organisation qui le conçoit ; une équipe, un microservice



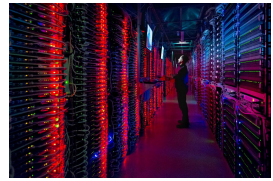
un *datacenter* de *cloud*
photo : Infopro Digital, droits réservés

†. M. E. Conway. « How Do Committees Invent ? » ↗. *Datamation* (1968).

Les années 2010-2020 : l'hyperspécialisation

1. Des équipes de développeurs aux équipes d'agents

- le *cloud*, les microservices, l'intégration et le déploiement continus
- les rôles éclatent : *frontend*, *backend*, mobile, DevOps, SRE, données, QA, SecOps ; côté produit, *Product Owner*, UX, *Scrum Master*
- la loi de Conway † : un système copie la structure de communication de l'organisation qui le conçoit ; une équipe, un microservice
- F. Brooks ‡ : n personnes, $n(n-1)/2$ canaux de communication ; **la coordination devient le goulot d'étranglement**



un *datacenter* de *cloud*
photo : Infopro Digital, droits réservés

†. M. E. Conway. « How Do Committees Invent ? ». *Datamation* (1968).

‡. F. P. Brooks Jr. *The Mythical Man-Month : Essays on Software Engineering*. Addison-Wesley, 1975.

Les années 2010-2020 : l'hyperspécialisation

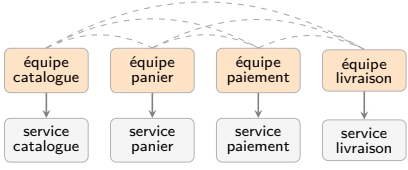
1. Des équipes de développeurs aux équipes d'agents

- le *cloud*, les microservices, l'intégration et le déploiement continus
- les rôles éclatent : *frontend*, *backend*, mobile, DevOps, SRE, données, QA, SecOps ; côté produit, *Product Owner*, UX, *Scrum Master*
- la loi de Conway † : un système copie la structure de communication de l'organisation qui le conçoit ; une équipe, un microservice
- F. Brooks ‡ : n personnes, $n(n-1)/2$ canaux de communication ; **la coordination devient le goulot d'étranglement**



un *datacenter* de *cloud*
photo : Infopro Digital, droits réservés

4 équipes, 6 canaux de communication



†. M. E. Conway. « How Do Committees Invent ? » ↗. *Datamation* (1968).

‡. F. P. Brooks Jr. *The Mythical Man-Month : Essays on Software Engineering*. Addison-Wesley, 1975.

rôle	ce qu'il fait	ce qu'il produit
product owner*, *product manager	décide *quoi* construire, pour qui	la spécification, les cas d'usage
développeur (*coder*)	implémente la spécification	le code
testeur (QA)	écrit les tests à partir de la spécification, pas du code	les tests, un rapport de tests
relecteur (*reviewer*)	relit chaque changement avant la fusion	des commentaires de revue
DevOps	intègre, déploie, exploite	l'intégration continue, la configuration

rôle	ce qu'il fait	ce qu'il produit
product owner*, *product manager	décide *quoi* construire, pour qui	la spécification, les cas d'usage
développeur (*coder*)	implémente la spécification	le code
testeur (QA)	écrit les tests à partir de la spécification, pas du code	les tests, un rapport de tests
relecteur (*reviewer*)	relit chaque changement avant la fusion	des commentaires de revue
DevOps	intègre, déploie, exploite	l'intégration continue, la configuration

- chaque rôle a ses **artefacts** : c'est par eux que l'équipe se coordonne

rôle	ce qu'il fait	ce qu'il produit
product owner*, *product manager	décide *quoi* construire, pour qui	la spécification, les cas d'usage
développeur (*coder*)	implémente la spécification	le code
testeur (QA)	écrit les tests à partir de la spécification, pas du code	les tests, un rapport de tests
relecteur (*reviewer*)	relit chaque changement avant la fusion	des commentaires de revue
DevOps	intègre, déploie, exploite	l'intégration continue, la configuration

- chaque rôle a ses **artefacts** : c'est par eux que l'équipe se coordonne
- en section 2, chacun de ces rôles devient un **agent**

années 1980 — années 1990-2000 — années 2010-2020 — **demain**

- les agents prennent les rôles spécialisés : coder, tester, intégrer, documenter



des agents de code (cours 3)

années 1980 — années 1990-2000 — années 2010-2020 — **demain**

- les agents prennent les rôles spécialisés : coder, tester, intégrer, documenter
- le développeur redevient **généraliste** : il orchestre des agents spécialisés (section 2) ; une petite équipe livre ce qui demandait une équipe entière



des agents de code (cours 3)

années 1980 — années 1990-2000 — années 2010-2020 — **demain**

- les agents prennent les rôles spécialisés : coder, tester, intégrer, documenter
- le développeur redevient **généraliste** : il orchestre des agents spécialisés (section 2) ; une petite équipe livre ce qui demandait une équipe entière
- Conway, encore : dans une équipe d'agents, chaque agent possède ses fichiers (section 2)



des agents de code (cours 3)

années 1980 — années 1990-2000 — années 2010-2020 — **demain**

- les agents prennent les rôles spécialisés : coder, tester, intégrer, documenter
- le développeur redevient **généraliste** : il orchestre des agents spécialisés (section 2) ; une petite équipe livre ce qui demandait une équipe entière
- Conway, encore : dans une équipe d'agents, chaque agent possède ses fichiers (section 2)
- mais ce n'est pas le retour de l'artisan : le développeur se place **aux bords de la boucle**, responsable de l'intention et de la vérification



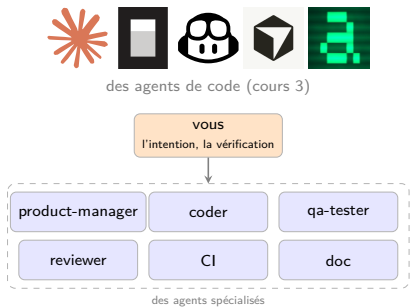
des agents de code (cours 3)

Demain : le chef d'orchestre

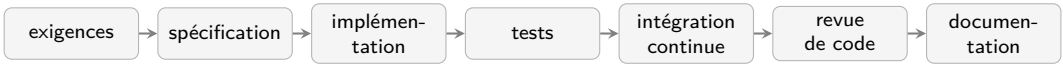
1. Des équipes de développeurs aux équipes d'agents



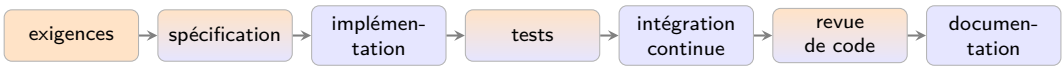
- les agents prennent les rôles spécialisés : coder, tester, intégrer, documenter
- le développeur redevient **généraliste** : il orchestre des agents spécialisés (section 2) ; une petite équipe livre ce qui demandait une équipe entière
- Conway, encore : dans une équipe d'agents, chaque agent possède ses fichiers (section 2)
- mais ce n'est pas le retour de l'artisan : le développeur se place **aux bords de la boucle**, responsable de l'intention et de la vérification



Le cycle de vie d'une tâche : qui fait quoi ?

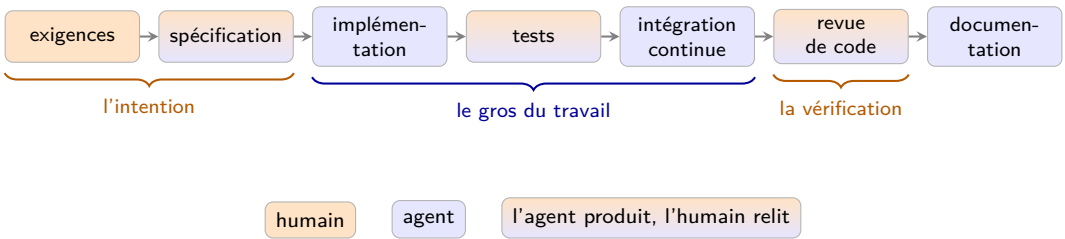


Le cycle de vie d'une tâche : qui fait quoi ?



humain agent l'agent produit, l'humain relit

Le cycle de vie d'une tâche : qui fait quoi ?



- l'humain se place **aux bords** de la boucle : l'intention au début, la vérification à la fin
- tout ce cours vise à rendre ces deux bords solides

1 Des équipes de développeurs aux équipes d'agents

2 Les systèmes multi-agents : coordonner, communiquer
Coordonner : qui fait quoi
Communiquer : comment les agents échangent
Retours d'expérience

3 Les méthodes formelles : vérifier ce qu'écrit la machine

4 Votre plus-value, quand la machine écrit le code

5 Synthèse

- un agent avec un énorme contexte se disperse et oublie ses consignes (cours 3 : le contexte est sa mémoire de travail)

- un agent avec un énorme contexte se disperse et oublie ses consignes (cours 3 : le contexte est sa mémoire de travail)
- **la spécialisation** : chaque rôle a son *system prompt*, ses outils, son contexte (une ou des *skills* par rôle)

- un agent avec un énorme contexte se disperse et oublie ses consignes (cours 3 : le contexte est sa mémoire de travail)
- **la spécialisation** : chaque rôle a son *system prompt*, ses outils, son contexte (une ou des *skills* par rôle)
- **la séparation des rôles** : l'agent qui écrit le code ne doit pas écrire les tests qui le jugent (cours 3 : l'oracle hors de portée de l'agent qu'il juge)

- un agent avec un énorme contexte se disperse et oublie ses consignes (cours 3 : le contexte est sa mémoire de travail)
- **la spécialisation** : chaque rôle a son *system prompt*, ses outils, son contexte (une ou des *skills* par rôle)
- **la séparation des rôles** : l'agent qui écrit le code ne doit pas écrire les tests qui le jugent (cours 3 : l'oracle hors de portée de l'agent qu'il juge)
- **le parallélisme** : des tâches indépendantes, en même temps

- un agent avec un énorme contexte se disperse et oublie ses consignes (cours 3 : le contexte est sa mémoire de travail)
- **la spécialisation** : chaque rôle a son *system prompt*, ses outils, son contexte (une ou des *skills* par rôle)
- **la séparation des rôles** : l'agent qui écrit le code ne doit pas écrire les tests qui le jugent (cours 3 : l'oracle hors de portée de l'agent qu'il juge)
- **le parallélisme** : des tâches indépendantes, en même temps
- le prix : plus de *tokens* (chaque agent relit le projet), et des problèmes de coordination, comme dans tout système réparti

Un système multi-agents est une organisation

- une vieille idée : la méthode Gaia [†] conçoit un système multi-agents comme une **organisation**

†. F. Zambonelli et al. « Developing Multiagent Systems : The Gaia Methodology » ↗. *ACM Transactions on Software Engineering and Methodology* (2003).

Un système multi-agents est une organisation

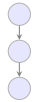
- une vieille idée : la méthode Gaia [†] conçoit un système multi-agents comme une **organisation**
- des **rôles**, chacun avec :
  - des responsabilités : ce qu'il doit faire arriver (vivacité), ce qu'il doit empêcher (sûreté)
  - des permissions : ce qu'il peut lire et modifier ; chez nous, une *skill* et les `permissions` d'opencode

†. F. Zambonelli et al. « Developing Multiagent Systems : The Gaia Methodology ». *ACM Transactions on Software Engineering and Methodology* (2003).

Un système multi-agents est une organisation

- une vieille idée : la méthode Gaia † conçoit un système multi-agents comme une **organisation**
- des **rôles**, chacun avec :
  - des responsabilités : ce qu'il doit faire arriver (vivacité), ce qu'il doit empêcher (sûreté)
  - des permissions : ce qu'il peut lire et modifier ; chez nous, une *skill* et les `permissions` d'opencode
- des **protocoles** : qui interagit avec qui, et comment
- des **règles**, des contraintes globales ; chez nous, `AGENTS.md`
- une **structure** : la topologie, et qui décide ; c'est l'architecture (transparent suivant)

†. F. Zambonelli et al. « Developing Multiagent Systems : The Gaia Methodology » ↗. *ACM Transactions on Software Engineering and Methodology* (2003).



la chaîne



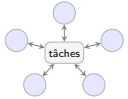
le routeur



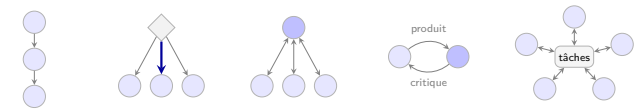
superviseur
et ouvriers



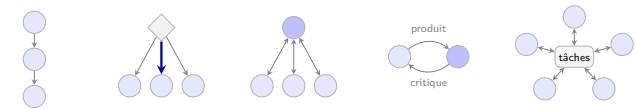
l'évaluateur
et l'optimiseur



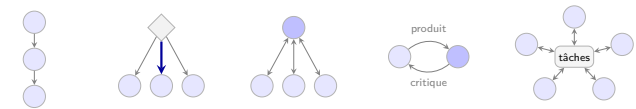
l'essaim



- **la chaîne** : chaque agent part du résultat du précédent (le *backend* décrit son API, le *frontend* s'en sert)



- **la chaîne** : chaque agent part du résultat du précédent (le *backend* décrit son API, le *frontend* s'en sert)
- **le routeur** : un classifieur léger envoie chaque tâche au bon spécialiste
- **superviseur et ouvriers** : un *lead* découpe la tâche, la délègue, relit les résultats
- **l'évaluateur et l'optimiseur** : l'un produit, l'autre critique, en boucle ; le *coder* et le *reviewer*



- **la chaîne** : chaque agent part du résultat du précédent (le *backend* décrit son API, le *frontend* s'en sert)
- **le routeur** : un classifieur léger envoie chaque tâche au bon spécialiste
- **superviseur et ouvriers** : un *lead* découpe la tâche, la délègue, relit les résultats
- **l'évaluateur et l'optimiseur** : l'un produit, l'autre critique, en boucle ; le *coder* et le *reviewer*
- **l'essaim** : des pairs, sans chef

Superviseur et ouvriers, en pratique : les sous-agents

- un agent délègue une tâche à un **sous-agent**, par un outil : `subagent(role, tâche)`

Superviseur et ouvriers, en pratique : les sous-agents

- un agent délègue une tâche à un **sous-agent**, par un outil : `subagent(role, tâche)`
- le sous-agent part d'un **contexte neuf** : il reçoit la tâche, pas toute la conversation

Superviseur et ouvriers, en pratique : les sous-agents

- un agent délègue une tâche à un **sous-agent**, par un outil : `subagent(role, tâche)`
- le sous-agent part d'un **contexte neuf** : il reçoit la tâche, pas toute la conversation
- il rend un court rapport, pas ses essais et ses erreurs : le contexte du *lead* reste petit

Superviseur et ouvriers, en pratique : les sous-agents

- un agent délègue une tâche à un **sous-agent**, par un outil : `subagent(role, tâche)`
- le sous-agent part d'un **contexte neuf** : il reçoit la tâche, pas toute la conversation
- il rend un court rapport, pas ses essais et ses erreurs : le contexte du *lead* reste petit
- en pratique, un sous-agent se déclare dans un fichier : une description (pour que le *lead* sache quand l'appeler), un *prompt*, ses permissions

`.opencode/agents/qa-tester.md`, avec opencode

```
---
description: Writes unit tests
  from SPEC.md
mode: subagent
permissions:
  - action: read
    resource: "*src/*"
    effect: deny
---
You write tests from the
specification only.
```

la règle deny *impose* que le testeur ne lise pas le code : c'est une permission, que le *harness* fait respecter, pas une consigne

→ démo : `demo_delegate.py`, la délégation à un sous-agent

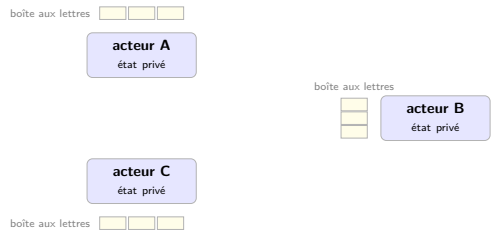
- **la mémoire partagée** : les processus lisent et écrivent les mêmes données, et se synchronisent
  - des verrous, des sémaphores (E. W. Dijkstra, 1965), des moniteurs (C. A. R. Hoare, 1974)
  - un espace commun où chacun lit et écrit : l'espace de tuples de Linda (D. Gelernter, 1985), le tableau noir partagé de Hearsay-II (L. D. Erman et al., 1980)

- **la mémoire partagée** : les processus lisent et écrivent les mêmes données, et se synchronisent
  - des verrous, des sémaphores (E. W. Dijkstra, 1965), des moniteurs (C. A. R. Hoare, 1974)
  - un espace commun où chacun lit et écrit : l'espace de tuples de Linda (D. Gelernter, 1985), le tableau noir partagé de Hearsay-II (L. D. Erman et al., 1980)
- **le passage de messages** : rien n'est partagé, les processus s'envoient des messages
  - les processus communicants (C. A. R. Hoare, 1978)
  - les acteurs (C. Hewitt, 1973)

- **la mémoire partagée** : les processus lisent et écrivent les mêmes données, et se synchronisent
  - des verrous, des sémaphores (E. W. Dijkstra, 1965), des moniteurs (C. A. R. Hoare, 1974)
  - un espace commun où chacun lit et écrit : l'espace de tuples de Linda (D. Gelernter, 1985), le tableau noir partagé de Hearsay-II (L. D. Erman et al., 1980)
- **le passage de messages** : rien n'est partagé, les processus s'envoient des messages
  - les processus communicants (C. A. R. Hoare, 1978)
  - les acteurs (C. Hewitt, 1973)
- et pour des agents ? Partager un dépôt, c'est de la mémoire partagée ; s'envoyer une tâche ou un rapport, c'est passer un message

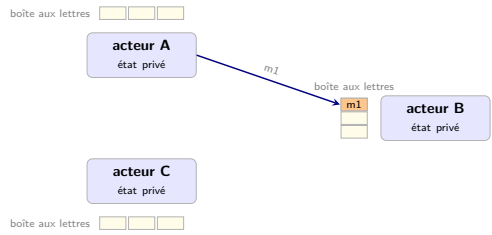
- **la mémoire partagée** : les processus lisent et écrivent les mêmes données, et se synchronisent
  - des verrous, des sémaphores (E. W. Dijkstra, 1965), des moniteurs (C. A. R. Hoare, 1974)
  - un espace commun où chacun lit et écrit : l'espace de tuples de Linda (D. Gelernter, 1985), le **tableau noir** partagé de Hearsay-II (L. D. Erman et al., 1980)
- **le passage de messages** : rien n'est partagé, les processus s'envoient des messages
  - les processus communicants (C. A. R. Hoare, 1978)
  - les **acteurs** (C. Hewitt, 1973)
- et pour des agents ? Partager un dépôt, c'est de la mémoire partagée ; s'envoyer une tâche ou un rapport, c'est passer un message

- C. Hewitt et al., 1973 †, déjà pour l'IA : un **acteur** a un état privé et une boîte aux lettres



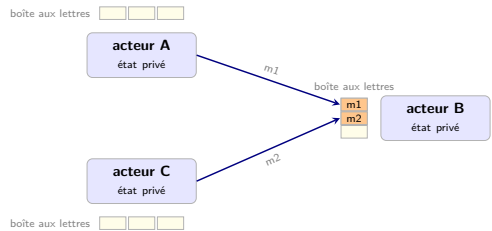
†. C. Hewitt et al. « A Universal Modular ACTOR Formalism for Artificial Intelligence » ↗. *IJCAI*. 1973.

- C. Hewitt et al., 1973 †, déjà pour l'IA : un **acteur** a un état privé et une boîte aux lettres
- on lui envoie des **messages**, de façon asynchrone : l'expéditeur n'attend pas



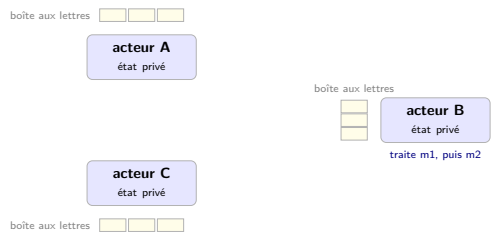
†. C. Hewitt et al. « A Universal Modular ACTOR Formalism for Artificial Intelligence » ↗. *IJCAI*. 1973.

- C. Hewitt et al., 1973 †, déjà pour l'IA : un **acteur** a un état privé et une boîte aux lettres
- on lui envoie des **messages**, de façon asynchrone : l'expéditeur n'attend pas



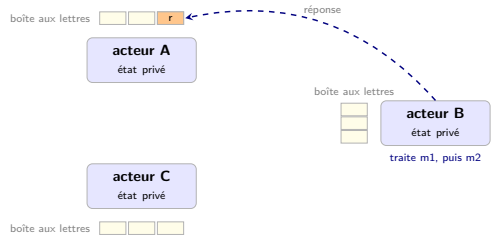
†. C. Hewitt et al. « A Universal Modular ACTOR Formalism for Artificial Intelligence » ↗. *IJCAI*. 1973.

- C. Hewitt et al., 1973 †, déjà pour l'IA : un **acteur** a un état privé et une boîte aux lettres
- on lui envoie des **messages**, de façon asynchrone : l'expéditeur n'attend pas
- il les traite **un à la fois**, et peut répondre par d'autres messages, déposés dans la boîte aux lettres du destinataire



†. C. Hewitt et al. « A Universal Modular ACTOR Formalism for Artificial Intelligence » ↗. *IJCAI*. 1973.

- C. Hewitt et al., 1973 †, déjà pour l'IA : un **acteur** a un état privé et une boîte aux lettres
- on lui envoie des **messages**, de façon asynchrone : l'expéditeur n'attend pas
- il les traite **un à la fois**, et peut répondre par d'autres messages, déposés dans la boîte aux lettres du destinataire
- aucune mémoire partagée : pas de course aux données, pas de verrou ; un acteur peut être n'importe où sur le réseau



†. C. Hewitt et al. « A Universal Modular ACTOR Formalism for Artificial Intelligence ». *IJCAI*. 1973.

Un agent est un acteur

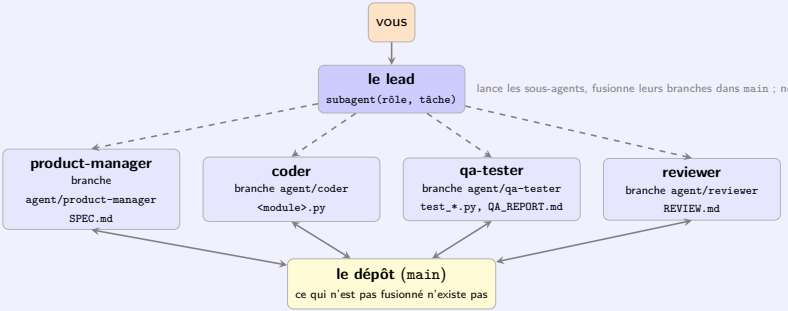
- son contexte est son état privé : aucun autre agent ne le lit
- une tâche, un fichier (un artefact), un rapport est un message
- le *lead* et ses ouvriers n'échangent que des messages : la tâche à l'aller, le rapport au retour

- son contexte est son état privé : aucun autre agent ne le lit
- une tâche, un fichier (un artefact), un rapport est un message
- le *lead* et ses ouvriers n'échangent que des messages : la tâche à l'aller, le rapport au retour

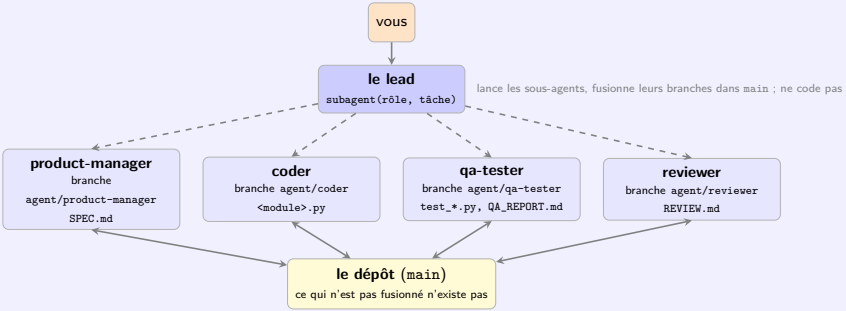
des canevas (*frameworks*) multi-agents à acteurs

- **AutoGen** (Microsoft) : des agents acteurs, des messages asynchrones ; réparti avec Orleans
- **Dapr Agents** (CNCF) : un acteur « virtuel » par agent ; les inactifs sont libérés, leur état conservé
- **Akka** (*Agentic Platform*) : des agents sur des acteurs en grappe, qui délèguent ou partagent une liste de tâches

→ projet : `README.md` : une équipe d'agents qui ne communique que par le dépôt

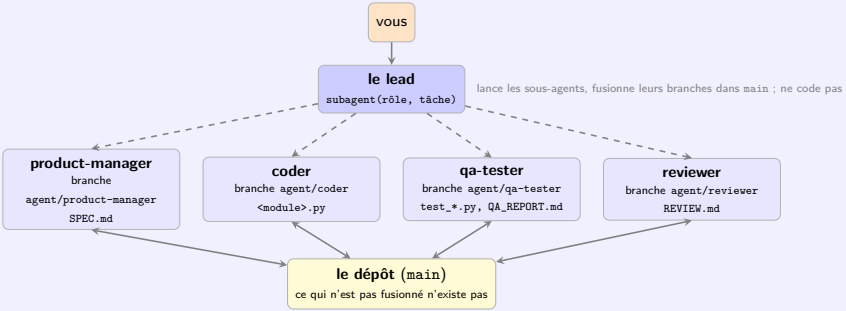


→ projet : `README.md` : une équipe d'agents qui ne communique que par le dépôt



- le dépôt sert de **tableau noir** collaboratif : chaque agent y dépose son travail, sur sa branche, et y lit celui des autres, sur `main`

→ projet : `README.md` : une équipe d'agents qui ne communique que par le dépôt



- le dépôt sert de **tableau noir** collaboratif : chaque agent y dépose son travail, sur sa branche, et y lit celui des autres, sur `main`
- `AGENTS.md` définit **qui fait quoi** : le rôle de chaque agent, les fichiers qu'il possède, et qui fusionne les branches dans `main` (le *lead*, seul)

- **chercher des vulnérabilités** † : des agents indépendants, chacun sur son répertoire, en trouvent 21 (6,5 millions de *tokens*) ; un essaim qui se coordonne, 266 (27 millions), dont seulement 12 en commun

†. Anthropic. *Patterns and problems in emerging multiagent systems*↗. anthropic.com. 2026.

- **chercher des vulnérabilités** † : des agents indépendants, chacun sur son répertoire, en trouvent 21 (6,5 millions de *tokens*) ; un essaim qui se coordonne, 266 (27 millions), dont seulement 12 en commun
- **écrire un compilateur C** : 16 agents, deux semaines, près de 2 000 sessions, un peu moins de 20 000 $; 100 000 lignes de Rust, qui compilent Linux 6.9 (x86, ARM, RISC-V)

†. Anthropic. *Patterns and problems in emerging multiagent systems*↗. anthropic.com. 2026.

Des réussites

- **chercher des vulnérabilités** † : des agents indépendants, chacun sur son répertoire, en trouvent 21 (6,5 millions de *tokens*) ; un essaim qui se coordonne, 266 (27 millions), dont seulement 12 en commun
- **écrire un compilateur C** : 16 agents, deux semaines, près de 2 000 sessions, un peu moins de 20 000 $; 100 000 lignes de Rust, qui compilent Linux 6.9 (x86, ARM, RISC-V)
- **chercher de l'information** : un *lead* et des sous-agents font 90,2 % mieux qu'un agent seul, mais consomment environ 15 fois plus de *tokens* qu'une conversation

  → démo : `demo_parallel.py`, une recherche séquentielle, puis parallèle

†. Anthropic. *Patterns and problems in emerging multiagent systems*↗. anthropic.com. 2026.

- **chercher des vulnérabilités** † : des agents indépendants, chacun sur son répertoire, en trouvent 21 (6,5 millions de *tokens*) ; un essaim qui se coordonne, 266 (27 millions), dont seulement 12 en commun
- **écrire un compilateur C** : 16 agents, deux semaines, près de 2 000 sessions, un peu moins de 20 000 $; 100 000 lignes de Rust, qui compilent Linux 6.9 (x86, ARM, RISC-V)
- **chercher de l'information** : un *lead* et des sous-agents font 90,2 % mieux qu'un agent seul, mais consomment environ 15 fois plus de *tokens* qu'une conversation

  → démo : `demo_parallel.py`, une recherche séquentielle, puis parallèle
- le point commun : des tâches **parallélisables**, et un **oracle** solide

†. Anthropic. *Patterns and problems in emerging multiagent systems*↗. anthropic.com. 2026.

- MAST † (M. Cemri, . . ., M. Zaharia, J. Gonzalez, I. Stoica) : 1 642 exécutions annotées de 7 systèmes (MetaGPT, ChatDev. . .) ; de **41 à 86,7 %** d'échecs

†. M. Cemri et al. *Why Do Multi-Agent LLM Systems Fail ?*↗. arXiv :2503.13657. 2025.

- MAST † (M. Cemri, . . ., M. Zaharia, J. Gonzalez, I. Stoica) : 1 642 exécutions annotées de 7 systèmes (MetaGPT, ChatDev. . .) ; de **41 à 86,7 %** d'échecs

la conception du système 44 %	**le désalignement entre agents** 32 %	**la vérification** 24 %
étapes répétées 15,7 % fin mal connue 12,4 % consignes ignorées 11,8 %	raisonnement et action divergent 13,2 % la tâche dérive 7,4 %	vérification fausse 9,1 % absente ou partielle 8,2 %

†. M. Cemri et al. *Why Do Multi-Agent LLM Systems Fail ?*↗. arXiv :2503.13657. 2025.

- MAST † (M. Cemri, . . ., M. Zaharia, J. Gonzalez, I. Stoica) : 1 642 exécutions annotées de 7 systèmes (MetaGPT, ChatDev. . .) ; de **41 à 86,7 %** d'échecs

la conception du système 44 %	**le désalignement entre agents** 32 %	**la vérification** 24 %
étapes répétées 15,7 % fin mal connue 12,4 % consignes ignorées 11,8 %	raisonnement et action divergent 13,2 % la tâche dérive 7,4 %	vérification fausse 9,1 % absente ou partielle 8,2 %

- les échecs viennent surtout de la **conception** et de la **vérification**, pas seulement du modèle : « *good MAS design requires organizational understanding* »

†. M. Cemri et al. *Why Do Multi-Agent LLM Systems Fail ?*↗. arXiv :2503.13657. 2025.

- MAST † (M. Cemri, . . ., M. Zaharia, J. Gonzalez, I. Stoica) : 1 642 exécutions annotées de 7 systèmes (MetaGPT, ChatDev. . .) ; de **41 à 86,7 %** d'échecs

la conception du système 44 %	**le désalignement entre agents** 32 %	**la vérification** 24 %
étapes répétées 15,7 % fin mal connue 12,4 % consignes ignorées 11,8 %	raisonnement et action divergent 13,2 % la tâche dérive 7,4 %	vérification fausse 9,1 % absente ou partielle 8,2 %

- les échecs viennent surtout de la **conception** et de la **vérification**, pas seulement du modèle : « *good MAS design requires organizational understanding* »
- mieux spécifier les rôles : +9,4 points ; ajouter une vérification : +15,6 points

†. M. Cemri et al. *Why Do Multi-Agent LLM Systems Fail ?*↗. arXiv :2503.13657. 2025.

À retenir

1. plusieurs agents : des contextes petits et spécialisés, et des rôles séparés
2. **coordonner**, c'est concevoir une organisation (des rôles, des règles, une structure) ; de la chaîne à l'essaim, selon les dépendances entre les tâches
3. **communiquer** est un vieux problème des systèmes répartis : un agent est un acteur ; les agents s'échangent des artefacts, et le dépôt git est un bon bus
4. **l'oracle reste indépendant** : l'agent qui code n'écrit pas les tests qui le jugent
5. ce qui marche : des tâches parallélisables, avec un bon oracle ; les échecs viennent surtout de la conception et de la vérification (section 3) : la coordination **se conçoit**

1. Des équipes de développeurs aux équipes d'agents
2. Les systèmes multi-agents : coordonner, communiquer
3. Les méthodes formelles : vérifier ce qu'écrit la machine
4. Votre plus-value, quand la machine écrit le code
5. Synthèse

- cours 3 : l'oracle définit « fini » ; d'ordinaire, l'oracle, ce sont les tests

†. E. W. Dijkstra. *Notes on Structured Programming*↗. EWD249. 1970.

- cours 3 : l'oracle définit « fini » ; d'ordinaire, l'oracle, ce sont les tests
- un test vérifie **quelques exécutions** :

> *« Tester un programme peut montrer la présence de bogues, jamais leur absence. »*
>
> — E. W. Dijkstra †

†. E. W. Dijkstra. *Notes on Structured Programming*↗. EWD249. 1970.

- cours 3 : l'oracle définit « fini » ; d'ordinaire, l'oracle, ce sont les tests
- un test vérifie **quelques exécutions** :

  « Tester un programme peut montrer la présence de bogues, jamais leur absence. »

  — E. W. Dijkstra †
- la concurrence aggrave tout : un bogue peut n'apparaître que dans un entrelacement sur des millions

†. E. W. Dijkstra. *Notes on Structured Programming*↗. EWD249. 1970.

- cours 3 : l'oracle définit « fini » ; d'ordinaire, l'oracle, ce sont les tests
- un test vérifie **quelques exécutions** :

> *« Tester un programme peut montrer la présence de bogues, jamais leur absence. »*
>
> — E. W. Dijkstra †

- la concurrence aggrave tout : un bogue peut n'apparaître que dans un entrelacement sur des millions
- avec des agents, le volume de code croît plus vite que notre capacité à le relire

†. E. W. Dijkstra. *Notes on Structured Programming*↗. EWD249. 1970.

- cours 3 : l'oracle définit « fini » ; d'ordinaire, l'oracle, ce sont les tests
- un test vérifie **quelques exécutions** :

> *« Tester un programme peut montrer la présence de bogues, jamais leur absence. »*
>
> — E. W. Dijkstra †

- la concurrence aggrave tout : un bogue peut n'apparaître que dans un entrelacement sur des millions
- avec des agents, le volume de code croît plus vite que notre capacité à le relire
- les **méthodes formelles** : écrire *ce que* le système doit faire comme une spécification mathématique, et laisser un outil vérifier, **exhaustivement**, qu'il le fait

†. E. W. Dijkstra. *Notes on Structured Programming*↗. EWD249. 1970.

- les méthodes formelles existent depuis les années 1970, mais restaient réservées à quelques experts : spécifier et prouver coûte cher

†. M. Kleppmann. *Prediction : AI will make formal verification go mainstream*↗. martin.kleppmann.com. 2025.

- les méthodes formelles existent depuis les années 1970, mais restaient réservées à quelques experts : spécifier et prouver coûte cher
- l'argument de M. Kleppmann [†] :
  1. les LLM rendent bon marché l'écriture des spécifications et des preuves
  2. le code écrit par l'IA a besoin d'être vérifié, et la relecture humaine ne passe pas à l'échelle
  3. le vérificateur rejette toute hallucination : une preuve fausse ne passe pas, l'agent recommence

†. M. Kleppmann. *Prediction : AI will make formal verification go mainstream*↗. martin.kleppmann.com. 2025.

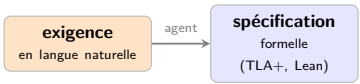
- les méthodes formelles existent depuis les années 1970, mais restaient réservées à quelques experts : spécifier et prouver coûte cher
- l'argument de M. Kleppmann [†] :
  1. les LLM rendent bon marché l'écriture des spécifications et des preuves
  2. le code écrit par l'IA a besoin d'être vérifié, et la relecture humaine ne passe pas à l'échelle
  3. le vérificateur rejette toute hallucination : une preuve fausse ne passe pas, l'agent recommence
- d'où sa prédiction : **l'IA va rendre la vérification formelle courante**

†. M. Kleppmann. *Prediction : AI will make formal verification go mainstream*↗. martin.kleppmann.com. 2025.

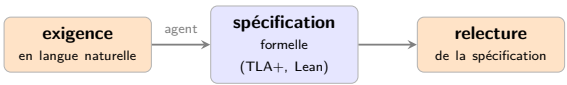
La machine spécifie, le vérificateur vérifie

3. Les méthodes formelles : vérifier ce qu'écrit la machine

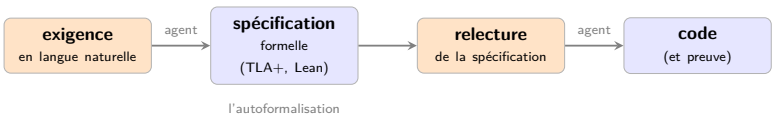
exigence
en langue naturelle



- **autoformalisation** : traduire la langue naturelle en un langage formel ; c'est l'agent qui écrit la spécification

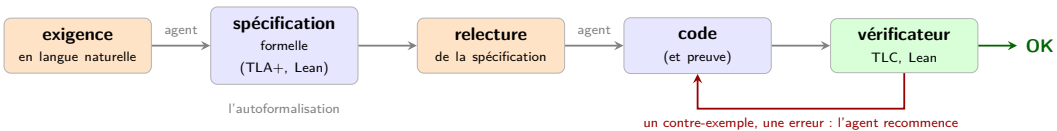


- **autoformalisation** : traduire la langue naturelle en un langage formel ; c'est l'agent qui écrit la spécification
- la spécification est courte et précise : **c'est la seule partie que vous devez lire**



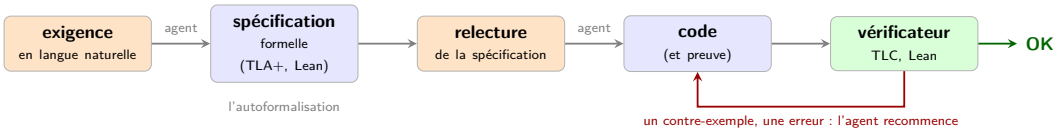
- **autoformalisation** : traduire la langue naturelle en un langage formel ; c'est l'agent qui écrit la spécification
- la spécification est courte et précise : **c'est la seule partie que vous devez lire**

La machine spécifie, le vérificateur vérifie



- **autoformalisation** : traduire la langue naturelle en un langage formel ; c'est l'agent qui écrit la spécification
- la spécification est courte et précise : **c'est la seule partie que vous devez lire**
- c'est le « tests d'abord » du cours 3, un cran plus haut : on relit la spécification *avant* de déléguer, puis l'agent itère contre un vérificateur qu'il ne peut pas tromper

La machine spécifie, le vérificateur vérifie



- **autoformalisation** : traduire la langue naturelle en un langage formel ; c'est l'agent qui écrit la spécification
- la spécification est courte et précise : **c'est la seule partie que vous devez lire**
- c'est le « tests d'abord » du cours 3, un cran plus haut : on relit la spécification *avant* de déléguer, puis l'agent itère contre un vérificateur qu'il ne peut pas tromper
- la confiance se déplace : non plus dans le code ni dans l'agent, mais dans **la spécification** (que vous lisez) et **le vérificateur** (petit, éprouvé)

- **TLA+** [†] (L. Lamport) : spécifier un système concurrent ou réparti comme une machine à états
  - les états initiaux, les pas possibles
  - les propriétés (**invariants**) que tout état atteignable doit vérifier

†. L. Lamport. *Specifying Systems : The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.

‡. C. Newcombe et al. « How Amazon Web Services Uses Formal Methods ». *Communications of the ACM* (2015).

- **TLA+** † (L. Lamport) : spécifier un système concurrent ou réparti comme une machine à états
  - les états initiaux, les pas possibles
  - les propriétés (**invariants**) que tout état atteignable doit vérifier
- **TLC**, son *model checker*, explore *tous* les entrelacements d'un modèle borné ; si une propriété est violée, il donne une **trace** qui y mène

†. L. Lamport. *Specifying Systems : The TLA+ Language and Tools for Hardware and Software Engineers*↗. Addison-Wesley, 2002.

‡. C. Newcombe et al. « How Amazon Web Services Uses Formal Methods »↗. *Communications of the ACM* (2015).

- **TLA+** † (L. Lamport) : spécifier un système concurrent ou réparti comme une machine à états
  - les états initiaux, les pas possibles
  - les propriétés (**invariants**) que tout état atteignable doit vérifier
- **TLC**, son *model checker*, explore *tous* les entrelacements d'un modèle borné ; si une propriété est violée, il donne une **trace** qui y mène
- utilisé dans l'industrie depuis longtemps : chez Amazon Web Services, il a trouvé des bogues subtils dans les conceptions de S3 et de DynamoDB ‡

†. L. Lamport. *Specifying Systems : The TLA+ Language and Tools for Hardware and Software Engineers*↗. Addison-Wesley, 2002.

‡. C. Newcombe et al. « How Amazon Web Services Uses Formal Methods »↗. *Communications of the ACM* (2015).

Exemple : une horloge

user

Écris une spécification TLA+ d'une horloge qui affiche les heures de 1 à 12, et avance d'une heure à chaque pas.

```
VARIABLE hr
HCini  == hr \in 1..12
HCnxt  == hr' = IF hr # 12 THEN hr + 1 ELSE 1
HC     == HCini /\ [][HCnxt]_hr
TypeOK == hr \in 1..12
```

l'horloge de L. Lamport †

l'état : l'heure `hr` ; un pas : l'heure suivante, et 1 après 12

la propriété `TypeOK` : l'heure reste toujours entre 1 et 12

le *model checker* (TLC) explore tous les états atteignables

†. L. Lamport. *Specifying Systems : The TLA+ Language and Tools for Hardware and Software Engineers*↗. Addison-Wesley, 2002.

Exemple : une horloge

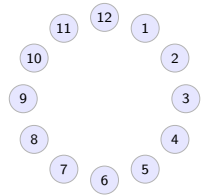
user

Écris une spécification TLA+ d'une horloge qui affiche les heures de 1 à 12, et avance d'une heure à chaque pas.

```
VARIABLE hr
HCini  == hr \in 1..12
HCnxt  == hr' = IF hr # 12 THEN hr + 1 ELSE 1
HC     == HCini /\ [][HCnxt]_hr
TypeOK == hr \in 1..12
```

l'horloge de L. Lamport †

l'état : l'heure `hr` ; un pas : l'heure suivante, et 1 après 12
la propriété `TypeOK` : l'heure reste toujours entre 1 et 12



le *model checker* (TLC) part des états initiaux : `HCini` permet n'importe quelle heure, de 1 à 12

†. L. Lamport. *Specifying Systems : The TLA+ Language and Tools for Hardware and Software Engineers*↗. Addison-Wesley, 2002.

Exemple : une horloge

user

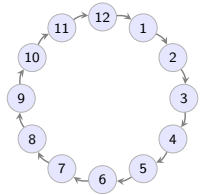
Écris une spécification TLA+ d'une horloge qui affiche les heures de 1 à 12, et avance d'une heure à chaque pas.

```
VARIABLE hr
HCini  == hr \in 1..12
HCnxt  == hr' = IF hr # 12 THEN hr + 1 ELSE 1
HC     == HCini /\ [][HCnxt]_hr
TypeOK == hr \in 1..12
```

l'horloge de L. Lamport †

l'état : l'heure `hr` ; un pas : l'heure suivante, et 1 après 12

la propriété `TypeOK` : l'heure reste toujours entre 1 et 12



puis il suit chaque pas : tous mènent à un état déjà vu ; 12 états, aucune erreur

†. L. Lamport. *Specifying Systems : The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.

Exemple : une horloge

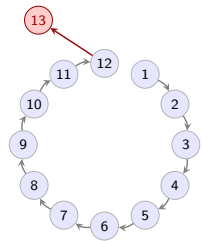
user

Écris une spécification TLA+ d'une horloge qui affiche les heures de 1 à 12, et avance d'une heure à chaque pas.

```
VARIABLE hr
HCini  == hr \in 1..12
HCnxt  == hr' = IF hr # 12 THEN hr + 1 ELSE 1
HC     == HCini /\ [][HCnxt]_hr
TypeOK == hr \in 1..12
```

l'horloge de L. Lamport †

l'état : l'heure `hr` ; un pas : l'heure suivante, et 1 après 12
la propriété `TypeOK` : l'heure reste toujours entre 1 et 12



si l'agent oublie le retour à 1 (`hr' = hr + 1`), 12 mène à 13 : `TypeOK` est violé

†. L. Lamport. *Specifying Systems : The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.

Un cas réel : Accord, dans Apache Cassandra

- Accord (CEP-15)↗ : les transactions d'Apache Cassandra
- Cassandra : une grande base de données répartie ; chez Apple, plus de 160 000 machines
- utilisée aussi par Netflix, Uber, Instagram, Walmart, Bloomberg, Activision, Blizzard…

Un cas réel : Accord, dans Apache Cassandra

- Accord (CEP-15)↗ : les transactions d'Apache Cassandra
- Cassandra : une grande base de données répartie ; chez Apple, plus de 160 000 machines
- utilisée aussi par Netflix, Uber, Instagram, Walmart, Bloomberg, Activision, Blizzard…
- Alexandre Siret, issu de la VAP ASR, a modélisé Accord en TLA+ : l'IA a généré une première version de la spécification, qu'il a corrigée, puis vérifiée avec TLC
- le *model checking* a renforcé la confiance dans le protocole



Community over Code, Glasgow, 12 octobre 2026

- **Lean** est à la fois un langage de programmation et un assistant de preuve
- une fonction, sa spécification (un théorème) et sa preuve vivent dans le même fichier ; le noyau de Lean vérifie la preuve

- **Lean** est à la fois un langage de programmation et un assistant de preuve
- une fonction, sa spécification (un théorème) et sa preuve vivent dans le même fichier ; le noyau de Lean vérifie la preuve
- la répartition des rôles :
  - **la preuve est le travail de l'agent** : elle est fastidieuse, et les messages d'erreur de Lean sont un retour parfait pour sa boucle ; une preuve acceptée par Lean n'a pas besoin d'être relue
  - **la spécification est le travail de l'humain** : la relire, car Lean prouve ce qui est écrit, pas ce que vous vouliez dire

- **Lean** est à la fois un langage de programmation et un assistant de preuve
- une fonction, sa spécification (un théorème) et sa preuve vivent dans le même fichier ; le noyau de Lean vérifie la preuve
- la répartition des rôles :
  - **la preuve est le travail de l'agent** : elle est fastidieuse, et les messages d'erreur de Lean sont un retour parfait pour sa boucle ; une preuve acceptée par Lean n'a pas besoin d'être relue
  - **la spécification est le travail de l'humain** : la relire, car Lean prouve ce qui est écrit, pas ce que vous vouliez dire
- d'autres outils sur la même idée, plus proches du code de tous les jours : Dafny, Verus (Rust), F*

Exemple : une horloge à 10^{12} heures

Et si l'horloge avait 10^{12} états ? TLC ne peut plus tous les explorer : il faut une preuve.

user

Écris en Lean une horloge à n heures, et prouve que l'heure reste entre 1 et n, pour tout n.

```
def next (n h : Nat) : Nat := if h < n then h + 1 else 1

theorem next_inRange (n h : Nat) (hn : 1 ≤ n) (hh : 1 ≤ h ∧ h ≤ n) :
    1 ≤ next n h ∧ next n h ≤ n := by unfold next; split <;> omega
theorem next_advances (n h : Nat) (hh : h < n) : next n h = h + 1 := by
  unfold next; simp [hh]
```

Exemple : une horloge à 10^{12} heures

Et si l'horloge avait 10^{12} états ? TLC ne peut plus tous les explorer : il faut une preuve.

user

Écris en Lean une horloge à n heures, et prouve que l'heure reste entre 1 et n, pour tout n.

```
def next (n h : Nat) : Nat := if h < n then h + 1 else 1

theorem next_inRange (n h : Nat) (hn : 1 ≤ n) (hh : 1 ≤ h ∧ h ≤ n) :
    1 ≤ next n h ∧ next n h ≤ n := by unfold next; split <;> omega
theorem next_advances (n h : Nat) (hh : h < n) : next n h = h + 1 := by
  unfold next; simp [hh]
```

- la preuve vaut **pour tout** n, 12 comme 10^{12} : Lean la vérifie sans explorer un seul état

Exemple : une horloge à 10^{12} heures

Et si l'horloge avait 10^{12} états ? TLC ne peut plus tous les explorer : il faut une preuve.

user

Écris en Lean une horloge à n heures, et prouve que l'heure reste entre 1 et n, pour tout n.

```
def next (n h : Nat) : Nat := if h < n then h + 1 else 1

theorem next_inRange (n h : Nat) (hn : 1 ≤ n) (hh : 1 ≤ h ∧ h ≤ n) :
    1 ≤ next n h ∧ next n h ≤ n := by unfold next; split <;> omega
theorem next_advances (n h : Nat) (hh : h < n) : next n h = h + 1 := by
  unfold next; simp [hh]
```

- la preuve vaut **pour tout** n, 12 comme 10^{12} : Lean la vérifie sans explorer un seul état
- le piège : une horloge bloquée sur 1 vérifie aussi `next_inRange` ; une **spécification trop faible**, qu'il faut relire. `next_advances` la complète : l'horloge avance

Exemple : une horloge à 10^{12} heures

Et si l'horloge avait 10^{12} états ? TLC ne peut plus tous les explorer : il faut une preuve.

user

Écris en Lean une horloge à n heures, et prouve que l'heure reste entre 1 et n, pour tout n.

```
def next (n h : Nat) : Nat := if h < n then h + 1 else 1

theorem next_inRange (n h : Nat) (hn : 1 ≤ n) (hh : 1 ≤ h ∧ h ≤ n) :
    1 ≤ next n h ∧ next n h ≤ n := by unfold next; split <;> omega
theorem next_advances (n h : Nat) (hh : h < n) : next n h = h + 1 := by
  unfold next; simp [hh]
```

- la preuve vaut **pour tout** n, 12 comme 10^{12} : Lean la vérifie sans explorer un seul état
- le piège : une horloge bloquée sur 1 vérifie aussi `next_inRange` ; une **spécification trop faible**, qu'il faut relire. `next_advances` la complète : l'horloge avance
- les preuves (après `by`) : écrites par l'agent, vérifiées par Lean

	tests	**TLA+ / TLC**	**Lean, Dafny, Verus**
ce qui est vérifié	quelques exécutions	tous les entrelacements d'une *conception* bornée	le *code*, pour toutes les entrées
usage typique	le code de tous les jours	protocoles concurrents, répartis	fonctions critiques, bibliothèques, cryptographie
ce que l'humain relit	les tests	le modèle et ses invariants	la spécification (les théorèmes)
coût	faible	moyen	élevé, mais en baisse grâce aux agents

- les méthodes formelles ne remplacent pas les tests : elles déplacent les vérifications les plus dures de « relire le code » à « relire la spécification »

À retenir

1. un test montre la présence de bogues, jamais leur absence ; un vérificateur couvre *tous* les cas de son modèle
2. la machine écrit la spécification (autoformalisation), le code et la preuve ; le vérificateur est l'oracle
3. **l'humain relit la spécification** : une spécification trop faible est prouvée tout aussi bien
4. TLA+ vérifie une conception concurrente ; Lean prouve du code

1. Des équipes de développeurs aux équipes d'agents
2. Les systèmes multi-agents : coordonner, communiquer
3. Les méthodes formelles : vérifier ce qu'écrit la machine
4. Votre plus-value, quand la machine écrit le code
5. Synthèse

- le développement passe d'un artisanat au clavier à la conduite d'une usine : vous concevez les chaînes, les oracles, les contrôles qualité ; les agents sont la chaîne de montage

- le développement passe d'un artisanat au clavier à la conduite d'une usine : vous concevez les chaînes, les oracles, les contrôles qualité ; les agents sont la chaîne de montage
- le goulot d'étranglement se déplace : de *écrire le code* à *savoir quoi construire*, et *vérifier que c'est juste*

- le développement passe d'un artisanat au clavier à la conduite d'une usine : vous concevez les chaînes, les oracles, les contrôles qualité ; les agents sont la chaîne de montage
- le goulot d'étranglement se déplace : de *écrire le code* à *savoir quoi construire*, et *vérifier que c'est juste*
- des systèmes qui s'entretiennent eux-mêmes : en production, ils lisent leurs erreurs, lancent un agent pour les diagnostiquer, proposent un correctif, passent l'intégration continue et le déploient

- le développement passe d'un artisanat au clavier à la conduite d'une usine : vous concevez les chaînes, les oracles, les contrôles qualité ; les agents sont la chaîne de montage
- le goulot d'étranglement se déplace : de *écrire le code* à *savoir quoi construire*, et *vérifier que c'est juste*
- des systèmes qui s'entretiennent eux-mêmes : en production, ils lisent leurs erreurs, lancent un agent pour les diagnostiquer, proposent un correctif, passent l'intégration continue et le déploient
- la sécurité passe de l'analyse du code à la surveillance, en direct, de ce que font les agents

- nous lirons peut-être de moins en moins de Python : le code devient ce que les agents utilisent pour commander les machines

- nous lirons peut-être de moins en moins de Python : le code devient ce que les agents utilisent pour commander les machines
- les humains lisent et écrivent des **spécifications** : des tests, des contrats, des spécifications formelles (section 3)

- nous lirons peut-être de moins en moins de Python : le code devient ce que les agents utilisent pour commander les machines
- les humains lisent et écrivent des **spécifications** : des tests, des contrats, des spécifications formelles (section 3)
- comme l'assembleur après l'arrivée des compilateurs : toujours là, rarement lu

- nous lirons peut-être de moins en moins de Python : le code devient ce que les agents utilisent pour commander les machines
- les humains lisent et écrivent des **spécifications** : des tests, des contrats, des spécifications formelles (section 3)
- comme l'assembleur après l'arrivée des compilateurs : toujours là, rarement lu
- ce qui prend de la valeur : le **jugement** et le **goût** (cours 1) ; savoir quoi construire, pour qui, et ce que « correct » veut dire

Si la machine écrit le code, ou une partie,
que vaut encore un ingénieur ?

- la question est légitime : dans certaines équipes, les agents écrivent déjà presque tout le code

Si la machine écrit le code, ou une partie,
que vaut encore un ingénieur ?

- la question est légitime : dans certaines équipes, les agents écrivent déjà presque tout le code
- notre réponse, dans la suite, est un **avis** : le nôtre, et il est résolument optimiste ; d'autres le sont moins (voir cours 1)

Si la machine écrit le code, ou une partie,
que vaut encore un ingénieur ?

- la question est légitime : dans certaines équipes, les agents écrivent déjà presque tout le code
- notre réponse, dans la suite, est un **avis** : le nôtre, et il est résolument optimiste ; d'autres le sont moins (voir cours 1)
- trois arguments :
  1. l'histoire : la destruction créatrice
  2. les données : où passe vraiment le temps d'un développeur
  3. ce que l'on attend désormais d'un ingénieur : le jugement, l'expertise, la responsabilité



J. Schumpeter, 1945
photo : L. F. Bachrach,
domaine public

« [. . .] the process of industrial mutation [. . .] that incessantly revolutionizes the economic structure from within, incessantly destroying the old one, incessantly creating a new one. This process of Creative Destruction is the essential fact about capitalism. »

— J. Schumpeter, 1942 †

- l'innovation détruit des métiers et en crée d'autres, d'un même mouvement

†. J. A. Schumpeter. *Capitalism, Socialism and Democracy.* Harper & Brothers, chap. 7. 1942.



J. Schumpeter, 1945
photo : L. F. Bachrach,
domaine public

« [. . .] the process of industrial mutation [. . .] that incessantly revolutionizes the economic structure from within, incessantly destroying the old one, incessantly creating a new one. This process of Creative Destruction is the essential fact about capitalism. »

— J. Schumpeter, 1942 †

- l'innovation détruit des métiers et en crée d'autres, d'un même mouvement
- la section 1 en est l'illustration : l'artisan a laissé place aux silos ; DBA, DevOps, *product owner* n'existaient pas avant leur époque

†. J. A. Schumpeter. *Capitalism, Socialism and Democracy*. Harper & Brothers, chap. 7. 1942.



J. Schumpeter, 1945
photo : L. F. Bachrach,
domaine public

« [. . .] the process of industrial mutation [. . .] that incessantly revolutionizes the economic structure from within, incessantly destroying the old one, incessantly creating a new one. This process of Creative Destruction is the essential fact about capitalism. »

— J. Schumpeter, 1942 †

- l'innovation détruit des métiers et en crée d'autres, d'un même mouvement
- la section 1 en est l'illustration : l'artisan a laissé place aux silos ; DBA, DevOps, *product owner* n'existaient pas avant leur époque
- les compilateurs n'ont pas supprimé les programmeurs : quand le logiciel coûte moins cher à écrire, on en écrit davantage

†. J. A. Schumpeter. *Capitalism, Socialism and Democracy.* Harper & Brothers, chap. 7. 1942.



J. Schumpeter, 1945
photo : L. F. Bachrach,
domaine public

« [. . .] the process of industrial mutation [. . .] that incessantly revolutionizes the economic structure from within, incessantly destroying the old one, incessantly creating a new one. This process of Creative Destruction is the essential fact about capitalism. »

— J. Schumpeter, 1942 †

- l'innovation détruit des métiers et en crée d'autres, d'un même mouvement
- la section 1 en est l'illustration : l'artisan a laissé place aux silos ; DBA, DevOps, *product owner* n'existaient pas avant leur époque
- les compilateurs n'ont pas supprimé les programmeurs : quand le logiciel coûte moins cher à écrire, on en écrit davantage
- **notre pari** : il en ira de même avec les agents ; le métier change, il ne disparaît pas

†. J. A. Schumpeter. *Capitalism, Socialism and Democracy.* Harper & Brothers, chap. 7. 1942.

- un développeur passe environ **14 %** de son temps à écrire du code (11 à 18 % selon les études) ; le reste : concevoir, relire, coordonner [†]

†. J. Butler et al. « Eight Myths on Software Engineering and GenAI » ↗. *ACM Queue* (2026).

- un développeur passe environ **14 %** de son temps à écrire du code (11 à 18 % selon les études) ; le reste : concevoir, relire, coordonner †
- accélérer le code accélère peu le reste, et la charge glisse vers l'étape suivante : pour un agent étudié, la moitié seulement des PR ont été fusionnées ; 15 % abandonnées, 15 % en attente d'un relecteur humain

†. J. Butler et al. « Eight Myths on Software Engineering and GenAI » ↗. *ACM Queue* (2026).

- un développeur passe environ **14 %** de son temps à écrire du code (11 à 18 % selon les études) ; le reste : concevoir, relire, coordonner †
- accélérer le code accélère peu le reste, et la charge glisse vers l'étape suivante : pour un agent étudié, la moitié seulement des PR ont été fusionnées ; 15 % abandonnées, 15 % en attente d'un relecteur humain
- 80 % des développeurs utilisent l'IA ; 29 % font confiance à ce qu'elle produit

†. J. Butler et al. « Eight Myths on Software Engineering and GenAI » ↗. *ACM Queue* (2026).

- un développeur passe environ **14 %** de son temps à écrire du code (11 à 18 % selon les études) ; le reste : concevoir, relire, coordonner †
- accélérer le code accélère peu le reste, et la charge glisse vers l'étape suivante : pour un agent étudié, la moitié seulement des PR ont été fusionnées ; 15 % abandonnées, 15 % en attente d'un relecteur humain
- 80 % des développeurs utilisent l'IA ; 29 % font confiance à ce qu'elle produit
- les gains varient selon la tâche, la personne, le contexte : l'impact de l'IA tient au **système de travail** autour du développeur, plus qu'au modèle

†. J. Butler et al. « Eight Myths on Software Engineering and GenAI » ↗. *ACM Queue* (2026).

Quand le code devient bon marché, le jugement devient le métier

- le code bon marché, c'est aussi le code fragile ou peu sûr bon marché : la valeur se déplace vers **vérifier, intégrer, exploiter** †
- ce qui compte : cadrer le problème, penser le système, vérifier et déboguer, la sécurité, la connaissance du domaine

†. B. Sharma. *When Code Becomes Cheap, Judgment Becomes the Craft*↗. bharatsharma.pro. 2026.

‡. S. Goedecke. *LLMs reward expertise*↗. seangoedecke.com. 2026.

Quand le code devient bon marché, le jugement devient le métier

- le code bon marché, c'est aussi le code fragile ou peu sûr bon marché : la valeur se déplace vers **vérifier, intégrer, exploiter** †
- ce qui compte : cadrer le problème, penser le système, vérifier et déboguer, la sécurité, la connaissance du domaine
- les LLM **récompensent l'expertise** ‡ :
  - « *The most important skill in prompting is expertise in the domain you're prompting for.* »
  - un expert sait quoi demander, et voit quand la réponse est fausse
  - « *For many tasks, the human is the bottleneck, not the model* » : le difficile est de spécifier la solution

†. B. Sharma. *When Code Becomes Cheap, Judgment Becomes the Craft*↗. bharatsharma.pro. 2026.

‡. S. Goedecke. *LLMs reward expertise*↗. seangoedecke.com. 2026.

Le code écrit par l'IA reste le vôtre

- écrire du code forçait à le comprendre ; quand un agent l'écrit, comprendre devient un coût à part, que l'on est tenté de sauter †
- posséder un logiciel, c'est répondre de sa qualité, de sa sécurité, de son exploitation, de son évolution : la question devient « voulons-nous posséder ceci ? »
- « *AI can be a complexity factory.* » : garder le logiciel compréhensible devient un objectif de conception

†. M. Lamb. *AI-Written Code Is Still Your Code. Are You OK With That?*↗. martiansoftware.com. 2026.

Le code écrit par l'IA reste le vôtre

- écrire du code forçait à le comprendre ; quand un agent l'écrit, comprendre devient un coût à part, que l'on est tenté de sauter †
- posséder un logiciel, c'est répondre de sa qualité, de sa sécurité, de son exploitation, de son évolution : la question devient « voulons-nous posséder ceci ? »
- « *AI can be a complexity factory.* » : garder le logiciel compréhensible devient un objectif de conception
- la séniorité, selon S. Gondala ‡ : la part d'un problème que l'on possède de bout en bout

junior	construit avec un agent, et sait voir quand il se trompe
senior	planifie et relit son propre travail
staff	prend un problème flou, arbitre, et répond du chiffre qui dit si ça a marché

†. M. Lamb. *AI-Written Code Is Still Your Code. Are You OK With That?*↗. martiansoftware.com. 2026.

‡. S. Gondala. *Building an AI-native engineering org*↗. x.com/sgondala2. 2026.

1. notre avis : la destruction créatrice, une fois de plus ; des métiers changent, d'autres naissent
2. écrire le code n'a jamais été tout le travail : le goulot passe à **décider quoi construire** et **vérifier** (Amdahl)
3. quand le code devient bon marché, **le jugement, l'expertise et la responsabilité** prennent de la valeur

1. Des équipes de développeurs aux équipes d'agents
2. Les systèmes multi-agents : coordonner, communiquer
3. Les méthodes formelles : vérifier ce qu'écrit la machine
4. Votre plus-value, quand la machine écrit le code
5. Synthèse

1. **l'humain passe aux bords de la boucle** : l'intention au début, la vérification à la fin
2. **diviser, et passer des artefacts** : des agents spécialisés, des contextes neufs, le dépôt pour communiquer
3. **garder l'oracle indépendant** : l'agent qui code n'écrit ni les tests ni la spécification qui le jugent
4. **les méthodes formelles déplacent la relecture vers la spécification** : la machine spécifie et prouve, le vérificateur vérifie, vous lisez la spécification
5. **votre plus-value** : le jugement, l'expertise, la responsabilité de ce que vous fusionnez

Le fil des quatre cours

Cours 1 l'*assistant* : le *prompt*, et l'humain qui boucle chaque itération

Cours 2 les *outils* : appel de fonctions, RAG, MCP ; le modèle agit, une étape à la fois

Cours 3 l'*agent* : la boucle se referme ; vous réglez le cadran de l'autonomie

Cours 4 l'*équipe* : plusieurs agents, des oracles plus forts ; vous concevez le système

Le fil des quatre cours

Cours 1 l'*assistant* : le *prompt*, et l'humain qui boucle chaque itération

Cours 2 les *outils* : appel de fonctions, RAG, MCP ; le modèle agit, une étape à la fois

Cours 3 l'*agent* : la boucle se referme ; vous réglez le cadran de l'autonomie

Cours 4 l'*équipe* : plusieurs agents, des oracles plus forts ; vous concevez le système

à chaque étape, la machine en fait plus ;
à chaque étape, vous restez responsable de ce qu'elle produit

- **en 2e année (S8)**, les bases pour juger ce que produit un agent :
  - **CSC4102**, génie logiciel orienté objet : spécifier, concevoir, préparer les tests de validation ; c'est l'oracle et l'intention, les deux bords de la boucle
  - **CSC4103**, programmation système : processus, appels système, *threads* et concurrence ; c'est le monde du bac à sable et de la concurrence

Et après ? Ce cours dans votre cursus

- **en 2e année (S8)**, les bases pour juger ce que produit un agent :
  - **CSC4102**, génie logiciel orienté objet : spécifier, concevoir, préparer les tests de validation ; c'est l'oracle et l'intention, les deux bords de la boucle
  - **CSC4103**, programmation système : processus, appels système, *threads* et concurrence ; c'est le monde du bac à sable et de la concurrence
- **les systèmes répartis** : une flotte d'agents en est un (acteurs, pannes partielles, vérification avec TLA+)
  - la VAP ASR (Architectes de Services informatiques Répartis)
  - le master PDS (Systèmes parallèles et distribués) de l'Institut Polytechnique de Paris

Et après ? Ce cours dans votre cursus

- **en 2e année (S8)**, les bases pour juger ce que produit un agent :
  - **CSC4102**, génie logiciel orienté objet : spécifier, concevoir, préparer les tests de validation ; c'est l'oracle et l'intention, les deux bords de la boucle
  - **CSC4103**, programmation système : processus, appels système, *threads* et concurrence ; c'est le monde du bac à sable et de la concurrence
- **les systèmes répartis** : une flotte d'agents en est un (acteurs, pannes partielles, vérification avec TLA+)
  - la VAP ASR (Architectes de Services informatiques Répartis)
  - le master PDS (Systèmes parallèles et distribués) de l'Institut Polytechnique de Paris
- **les modèles eux-mêmes** : les entraîner, les évaluer, les déployer
  - la VAP MAIA (Modèles et Applications de l'Intelligence Artificielle)
  - le master DataAI (Données et intelligence artificielle) de l'Institut Polytechnique de Paris

Les règles de chaque cours

- chaque cours fixe ses règles d'usage de l'IA : ce qui est permis, ce qui ne l'est pas, ce qu'il faut déclarer
- **respectez-les** ; en cas de doute, demandez à l'enseignant

- chaque cours fixe ses règles d'usage de l'IA : ce qui est permis, ce qui ne l'est pas, ce qu'il faut déclarer
- **respectez-les** ; en cas de doute, demandez à l'enseignant

Ne déléguez pas à la machine
le fait d'apprendre à réfléchir.